

PCP BC - Applications and Architecture

Course-End Project 1: Insurance Smart Contract

Name: Neeleshwari Mehra

Description

This project develops a decentralized, transparent, automated insurance system using Ethereum blockchain technology.

Problem Statement

The current insurance system is plagued with inefficiencies such as a lack of transparency, high administrative costs, and delays in claim processing. Manual handling of policy issuance, premium payments, claim submissions, and payouts can lead to errors, fraud, and slow service, ultimately impacting customer satisfaction. Policyholders face challenges in tracking their policies and claim statuses, while insurers struggle with managing and accessing claims promptly and accurately. A streamlined, transparent, and automated solution is essential to enhance the efficiency and reliability of the insurance process.

Solution

This project involves the development of a smart contract-based insurance system on the Ethereum blockchain. This decentralized application will automate the management of insurance policies, including policy issuance, premium payments, claim submissions, and payouts. Leveraging blockchain technology, the system will enhance transparency, reduce administrative overhead, and ensure that all transactions are secure and immutable. Smart contracts will automate key processes, reducing the risk of human error and fraud, and ensuring that claims are processed swiftly and accurately.

Project: Insurance Smart Contract:

1. REMIX IDE CODE

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract InsuranceContract {

    address public insurer;

    constructor() {
        insurer = msg.sender;
    }

    modifier onlyInsurer() {
        require(msg.sender == insurer, "Only insurer allowed");
        _;
    }

    struct Policy {
        address policyholder;
        uint premium;
        uint coverage;
        uint startDate;
        uint duration;
        bool active;
    }

    struct Claim {
        uint policyId;
        uint amount;
        string reason;
        bool approved;
        bool paid;
    }

    mapping(uint => Policy) public policies;
```

```

mapping(uint => Claim) public claims;

uint public policyCount;
uint public claimCount;

event PolicyIssued(
    uint policyId,
    address indexed policyholder,
    uint premium,
    uint coverage,
    uint duration
);

event PremiumPaid(
    uint policyId,
    address indexed policyholder,
    uint amount
);

event ClaimSubmitted(
    uint claimId,
    uint policyId,
    address indexed policyholder,
    uint amount,
    string reason
);

event ClaimApproved(
    uint claimId,
    uint policyId
);

event ClaimPaid(
    uint claimId,
    address indexed policyholder,
    uint amount
);

// ===== INSURER FUNCTIONS =====

```

```

function issuePolicy(
    address _policyholder,
    uint _premium,
    uint _coverage,
    uint _duration
) public onlyInsurer {

    policyCount++;

    policies[policyCount] = Policy({
        policyholder: _policyholder,
        premium: _premium,
        coverage: _coverage,
        startDate: block.timestamp,
        duration: _duration,
        active: true
    });

    emit PolicyIssued(
        policyCount,
        _policyholder,
        _premium,
        _coverage,
        _duration
    );
}

function approveClaim(uint _claimId) public onlyInsurer {
    Claim storage claim = claims[_claimId];
    require(!claim.approved, "Already approved");

    claim.approved = true;

    emit ClaimApproved(_claimId, claim.policyId);
}

function payClaim(uint _claimId) public onlyInsurer {
    Claim storage claim = claims[_claimId];
    require(claim.approved, "Claim not approved");
    require(!claim.paid, "Already paid");
}

```

```

        Policy storage policy = policies[claim.policyId];

        claim.paid = true;

        // payable
        (bool success, ) = payable(policy.policyholder).call{value:
claim.amount}("");
        require(success, "Payment failed");

        emit ClaimPaid(
            _claimId,
            policy.policyholder,
            claim.amount
        );
    }

    // ===== POLICYHOLDER FUNCTIONS =====

    function payPremium(uint _policyId) public payable {
        Policy storage policy = policies[_policyId];

        require(policy.active, "Policy inactive");
        require(msg.sender == policy.policyholder, "Not policyholder");
        require(msg.value == policy.premium, "Incorrect premium");

        emit PremiumPaid(
            _policyId,
            msg.sender,
            msg.value
        );
    }

    function submitClaim(
        uint _policyId,
        uint _amount,
        string memory _reason
    ) public {

        Policy storage policy = policies[_policyId];

```

```
require(policy.active, "Policy inactive");
require(msg.sender == policy.policyholder, "Not policyholder");
require(
    block.timestamp <= policy.startDate + policy.duration,
    "Policy expired"
);

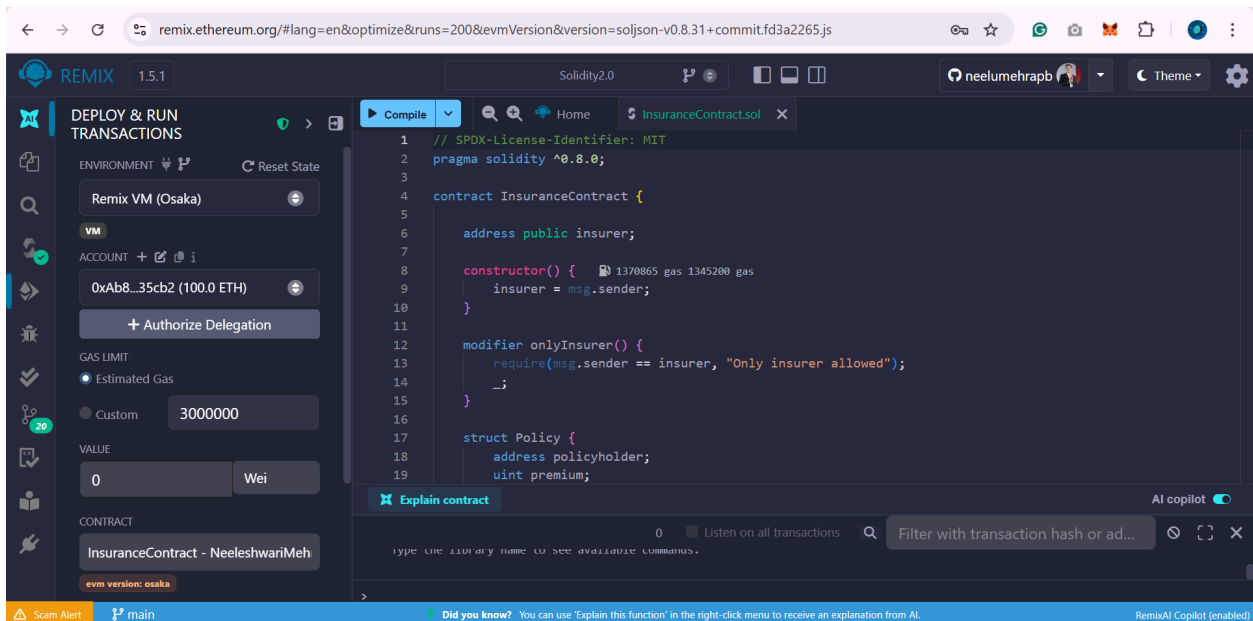
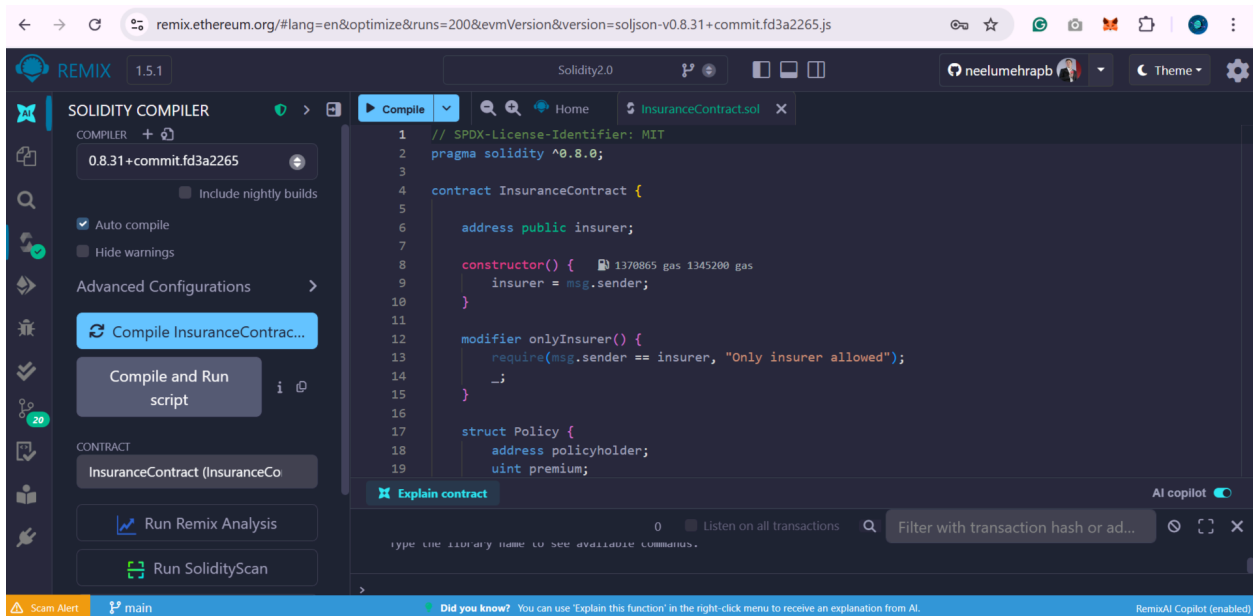
claimCount++;

claims[claimCount] = Claim({
    policyId: _policyId,
    amount: _amount,
    reason: _reason,
    approved: false,
    paid: false
});

emit ClaimSubmitted(
    claimCount,
    _policyId,
    msg.sender,
    _amount,
    _reason
);
}

receive() external payable {}
}
```

2. SCREENSHOT FOR DEBUG OUTPUT



remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.fd3a2265.js

REMIX 1.5.1 Solidity2.0 neelumehrapb Theme

DEPLOY & RUN TRANSACTIONS

Estimated Gas

Custom 3000000

VALUE 0 Wei

CONTRACT InsuranceContract - NeeleshwariMeh

evm version: osaka

Deploy

At Address Load contract from Address

Transactions recorded 1

Deployed Contracts 1

INSURANCECONTRACT AT 0xAc...

Compile Home InsuranceContract.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract InsuranceContract {
5
6     address public insurer;
7
8     constructor() {
9         1370865 gas 1345200 gas
10     }
11 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

creation of InsuranceContract pending...

[vm] from: 0xab8...35cb2 to: InsuranceContract.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xec6...813f4 Debug

Scam Alert main Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI. RemixAI Copilot (enabled)

remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.fd3a2265.js

REMIX 1.5.1 Solidity2.0 neelumehrapb Theme

FILE EXPLORER

+ Create

- .deps
- .git
- .states
- artifacts
- NeeleshwariMehra
 - InsuranceContract.sol
- workspace1
- .prettierrc.json
- ex5.sol
- ex6.sol
- ex7.sol
- README.md
- remix.config.json

Compile Home InsuranceContract.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract InsuranceContract {
5
6     address public insurer;
7
8     constructor() {
9         1370865 gas 1345200 gas
10         insurer = msg.sender;
11     }
12
13     modifier onlyInsurer() {
14         require(msg.sender == insurer, "Only insurer allowed");
15         _;
16     }
17
18     struct Policy {
19         address policyholder;
20         uint premium;
21     }
22 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

Type the library name to see available commands.

GIT main main

Scam Alert main Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI. RemixAI Copilot (enabled)

[illegible]

remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion=soljson-v0.8.31+commit.fd3a2265.js

REMIX 1.5.1 Solidity2.0 neelumehrapb Theme

DEPLOY & RUN TRANSACTIONS

Deployed Contracts 1

INSURANCECONTRACT AT 0XA

Balance: 0 ETH

approveClaim uint256_claimid

ISSUEPOLICY

_policyholder: 0x5B38Da6a701c568545dCfcB03Fc

_premium: 1

_coverage: 7

_duration: 3000000

Calldata Parameters transact

payClaim uint256_claimid

payPremium uint256_policyid

Compile Home InsuranceContract.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract InsuranceContract {
5
6     address public insurer;
7
8     constructor() { 1370865 gas 1345200 gas
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

creation of InsuranceContract pending...

[vm] from: 0xA8...35cb2 to: InsuranceContract.constructor() value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xec6...813f4

status 1 Transaction mined and execution succeed

transaction hash 0xec6bcb4e89c465287dd64d688328930576e86b87cddfa8de1a7b2ae79e813f4

block hash 0x274da69239ef8cf83f72a8741058ab27cc76dbf064376a2a9908662b11b72dec

Debug

Scam Alert main Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI. RemixAI Copilot (enabled)

remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion=soljson-v0.8.31+commit.fd3a2265.js

REMIX 1.5.1 Solidity2.0 neelumehrapb Theme

DEPLOY & RUN TRANSACTIONS

Balance: 0 ETH

approveClaim uint256_claimid

ISSUEPOLICY

_policyholder: 0x5B38Da6a701c568545dCfcB03Fc

_premium: 1

_coverage: 7

_duration: 3000000

Calldata Parameters transact

payClaim uint256_claimid

payPremium uint256_policyid

submitClaim uint256_policyid, uint256_

claimCount

claims uint256

Compile Home InsuranceContract.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract InsuranceContract {
5
6     address public insurer;
7
8     constructor() { 1370865 gas 1345200 gas
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0xA8...35cb2 to: InsuranceContract.issuePolicy(address,uint256,uint256) 0xA8...3510d value: 0 wei data: 0xA0B...dc6c0 logs: 1 hash: 0x0d4...d2b8c

status 1 Transaction mined and execution succeed

transaction hash 0x0d47f897e5b9d2ed33ba44c32813fedb8c3d8c2c69bccbdc4c9ed4708cd2b8c

block hash 0x7fe9c04a47fe16b05db834e22f301a5f486a3957e1af3243438123afb07c680

block number 47

from 0xA8B83F64d9C6d1EcF9b849Ae677d03315835cb2

to InsuranceContract.issuePolicy(address,uint256,uint256)

0xA8B83F64d9C6d1EcF9b849Ae677d03315835cb2

Debug

Scam Alert main Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI. RemixAI Copilot (enabled)

[illegible]

remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.fd3a2265.js

REMIX 1.5.1 Solidity2.0 neelumehrapb Theme

DEPLOY & RUN TRANSACTIONS

Compile Home InsuranceContract.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract InsuranceContract {
5
6     address public insurer;
7
8     constructor() { 1370865 gas 1345200 gas
```

Calldata Parameters **transact**

payClaim uint256_claimId

PAYPREMIUM

_policyId: 1

Calldata Parameters **transact**

submitClaim uint256_policyId, uint256_

claimCount

claims uint256

insurer

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: InsuranceContract.payPremium(uint256) 0xAc4...3510d value: 1 wei
data: 0x436...00001 logs: 1 hash: 0xc91...ea5ae Debug

status 1 Transaction mined and execution succeed

transaction hash 0xc911bcb2e14ef6f97d92b8a9dfed3ed89fb4f175d529bd900f941a590ea5ae

block hash 0xa489593784ff66da3b84a0cf080d0896de4e0797984a8961b12a66778fd0ac8c

block number 49

from 0x5B380a6a701c568545dcfc803fc8875f56beddC4

to InsuranceContract.payPremium(uint256) 0xAc4...3510d

Scam Alert main Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI. RemixAI Copilot (enabled)

remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.fd3a2265.js

REMIX 1.5.1 Solidity2.0 neelumehrapb Theme

DEPLOY & RUN TRANSACTIONS

Compile Home InsuranceContract.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract InsuranceContract {
5
6     address public insurer;
7
8     constructor() { 1370865 gas 1345200 gas
```

Calldata Parameters **transact**

payClaim uint256_claimId

PAYPREMIUM

_policyId: 1

_amount: 7

_reason: Hospital Expenses

Calldata Parameters **transact**

submitClaim uint256_policyId, uint256_

claimCount

claims uint256

insurer

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: InsuranceContract.submitClaim(uint256,uint256,string) 0xAc4...3510d
value: 0 wei data: 0x069...00000 logs: 1 hash: 0x906...4b9c5 Debug

status 1 Transaction mined and execution succeed

transaction hash 0x9067095052513e3d5e90688f08c9c7ace091aa4aed183d30d0da771ca4b9c5

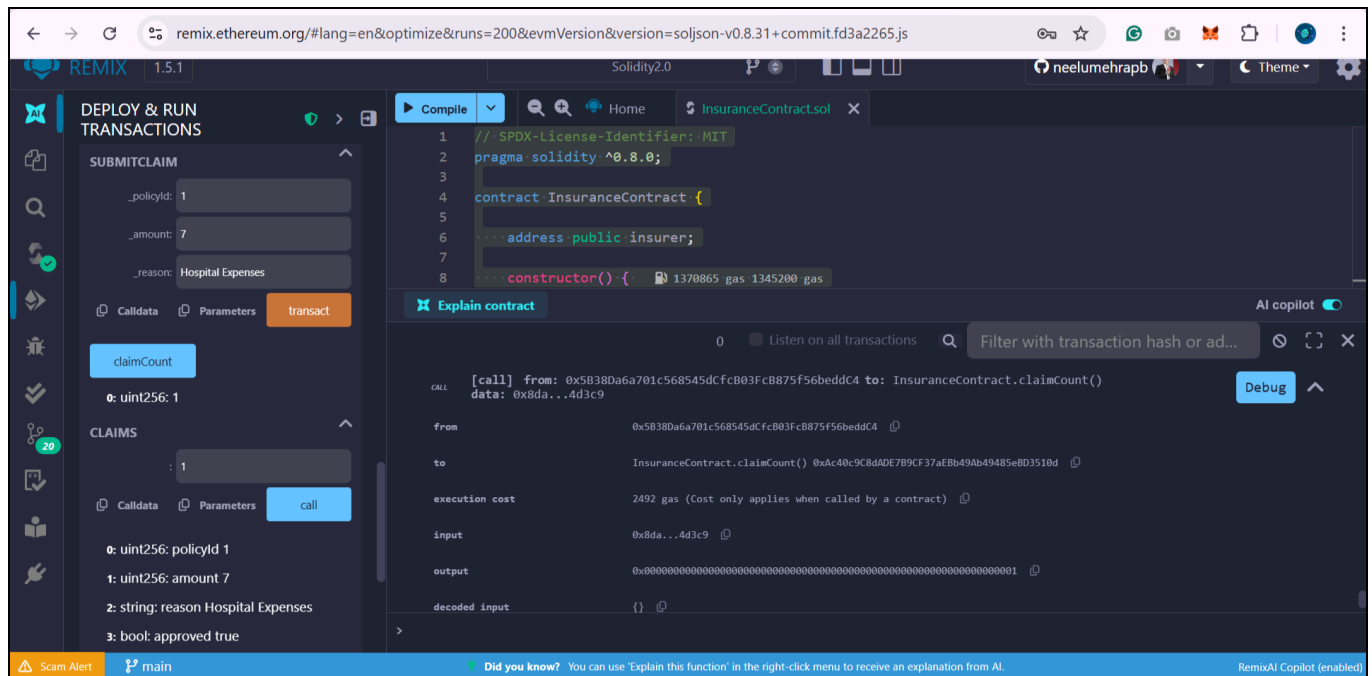
block hash 0xc33a452c15857e1d9066b8501c7e5a1f56ef95de37554782d9c68beed3ad51c

block number 50

from 0x5B380a6a701c568545dcfc803fc8875f56beddC4

to InsuranceContract.submitClaim(uint256,uint256,string) 0xAc40c9C8dADE789CF37aEBb49Ab49485eB03510d

Scam Alert main Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI. RemixAI Copilot (enabled)



3. DEBUG OUTPUT

```
[vm]
from: 0xAb8...35cb2
to: InsuranceContract.(constructor)
value: 0 wei
data: 0x608...f0033
logs: 0
hash: 0xec6...813f4
```

status	1 Transaction mined and execution succeed
transaction hash	0xec6bcd4e89c465287dd 64d688328930576e86b87c ddfa8de1a7b2ae79e813f4
block hash	0x274da69239ef8cf83f72 a8741058ab27cc76dbf064 376a2a9908662b11b72dec
block number	45

contract address	0xAc40c9C8dADE7B9CF37a EBb49Ab49485eBD3510d
from	0xAb8483F64d9C6d1EcF9b 849Ae677dD3315835cb2
to	InsuranceContract.(con structor)
transaction cost	1523549 gas
execution cost	1368765 gas


```
decoded input      {}
decoded output     -
logs               []
raw logs           []
```

```
>
```

```
[vm]
from: 0xAb8...35cb2
to:
InsuranceContract.issuePolicy(address,uint256,uint256,uint256)
0xAc4...3510d
value: 0 wei
data: 0xa0b...dc6c0
logs: 1
hash: 0x0d4...d2b8c
```

```
status              1 Transaction mined
                    and execution succeed

transaction hash     0x0d47f897e5b9d2ed33ba
                    44c32013fedb0c3d8c2c69
                    bccbcda4c9ed4708cd2b8c

block hash           0x7fefc04a47fef16b05db
                    834e22f301a5f486a3957e
                    1af3243438123afb07c680

block number         47

from                 0xAb8483F64d9C6d1EcF9b
                    849Ae677dD3315835cb2

to                  InsuranceContract.issu
                    ePolicy(address,uint25
                    6,uint256,uint256)
                    0xAc40c9C8dADE7B9CF37a
                    Ebb49Ab49485eBD3510d

transaction cost     183261 gas
execution cost       161385 gas
output               0x
```

decoded input

decoded output

logs

```
{
  "address
_policyholder":
"0x5B38Da6a701c568545d
CfcB03FcB875f56beddC4"
,
  "uint256
_premium": "1",
  "uint256
_coverage": "7",
  "uint256
_duration": "3000000"
}

{}

[
  {
    "from":
"0xAc40c9C8dADE7B9CF37
aEBb49Ab49485eBD3510d"
,
    "topic":
"0xd93ca75dcb625471848
5f83c72225dcd37a74fd9a
134ceed01946cf2e920b83
d",
    "event":
"PolicyIssued",
    "args": {
      "0":
"1",
      "1":
"0x5B38Da6a701c568545d
CfcB03FcB875f56beddC4"
,
      "2":
"1",
      "3":
"7",
      "4":
"3000000"

```

raw logs

```
    }
  }
]
[
  {
    "_type": "log",
    "address":
"0xAc40c9C8dADE7B9CF37
aEBb49Ab49485eBD3510d"
,
    "blockHash":
"0x7fefc04a47fef16b05d
b834e22f301a5f486a3957
e1af3243438123afb07c68
0",
    "blockNumber": 47,
    "data":
"0x00000000000000000000
0000000000000000000000
0000000000000000000000
1000000000000000000000
0000000000000000000000
0000000000000000000010
0000000000000000000000
0000000000000000000000
0000000000000000007000
0000000000000000000000
0000000000000000000000
0000000000000000000000
0000000000002dc6c0",
    "index": 1,
    "topics": [

"0xd93ca75dcb625471848
5f83c72225dcd37a74fd9a
134ceed01946cf2e920b83
d",

"0x00000000000000000000
000005b38da6a701c56854
5dcfcb03fcb875f56beddc
```

```
4"
    ],
    "transactionHash":
    "0x0d47f897e5b9d2ed33b
a44c32013fedb0c3d8c2c6
9bccbcda4c9ed4708cd2b8
c",

    "transactionIndex": 0
  }
]
```

>

```
[vm]
from: 0x5B3...eddC4
to: InsuranceContract.payPremium(uint256) 0xAc4...3510d
value: 1 wei
data: 0x436...00001
logs: 1
hash: 0xc91...ea5ae
```

status	1 Transaction mined and execution succeed
transaction hash	0xc911bcb2e14eff6f97d9 2b8a9dfed33ed89fb4f175 d5290d900f941a590ea5ae
block hash	0xa409593784ff66da3b84 a0cf080d0896de4e079798 4a8961b12a66778fd0ac8c
block number	49
from	0x5B38Da6a701c568545dC fCB03FcB875f56beddC4
to	InsuranceContract.payP remium(uint256) 0xAc40c9C8dADE7B9CF37a EBb49Ab49485eBD3510d
transaction cost	30080 gas

execution cost	8876 gas
output	0x
decoded input	<pre>{ "uint256 _policyId": "1" }</pre>
decoded output	<pre>{}</pre> <pre>[{ "from": "0xAc40c9C8dADE7B9CF37 aEBb49Ab49485eBD3510d" , "topic": "0xfbf1458fbbddabb878d e144276216e8ccda619cba c2d598e6ee37ff72173afb 1", "event": "PremiumPaid", "args": { "0": "1", "1": "0x5B38Da6a701c568545d CfcB03FcB875f56beddC4" , "2": "1" } }]</pre>
logs	

raw logs

```
[
  {
    "_type": "log",
    "address":
"0xAc40c9C8dADE7B9CF37
aEBb49Ab49485eBD3510d"
,
    "blockHash":
"0xa409593784ff66da3b8
4a0cf080d0896de4e07979
84a8961b12a66778fd0ac8
c",
    "blockNumber": 49,
    "data":
"0x00000000000000000000
0000000000000000000000
0000000000000000000000
1000000000000000000000
0000000000000000000000
0000000000000000000001"
,
    "index": 1,
    "topics": [

"0xfbf1458fbbddabb878d
e144276216e8ccda619cba
c2d598e6ee37ff72173afb
1",

"0x00000000000000000000
000005b38da6a701c56854
5dcfcb03fcb875f56beddc
4"

],
    "transactionHash":
"0xc911bcb2e14eff6f97d
92b8a9dfed33ed89fb4f17
5d5290d900f941a590ea5a
e",

"transactionIndex": 0
```

```
    }  
  ]  
  
value      1 wei
```

```
>
```

```
[vm]  
from: 0x5B3...eddC4  
to: InsuranceContract.submitClaim(uint256,uint256,string)  
0xAc4...3510d  
value: 0 wei  
data: 0x069...00000  
logs: 1  
hash: 0x906...4b9c5
```

```
status      1 Transaction mined  
            and execution succeed  
  
transaction hash 0x906e7095052513e3d5e9  
                0688f08c9c7ace091aa4ae  
                d1838d30d0da771ca4b9c5  
  
block hash  0xc33a452c15857e1d9066  
            b85501c7e5a1f56ef95de3  
            7554782d9c68beed3ad51c  
  
block number 50  
  
from          0x5B38Da6a701c568545dC  
            fcB03FcB875f56beddC4  
  
to           InsuranceContract.subm  
            itClaim(uint256,uint25  
            6,string)  
            0xAc40c9C8dADE7B9CF37a  
            Ebb49Ab49485eBD3510d  
  
transaction cost 127711 gas  
execution cost  105755 gas  
output          0x  
  
decoded input  {  
                "uint256  
                _policyId": "1",
```

decoded output

logs

```
        "uint256
_amount": "7",
        "string _reason":
"Hospital Expenses"
    }
    {}
    [
        {
            "from":
"0xAc40c9C8dADE7B9CF37
aEBb49Ab49485eBD3510d"
        ,
            "topic":
"0xd91ad459d6781ac1a94
eb5c677e1aa55ab8d256ca
e923e543a7e969235a041c
a",
            "event":
"ClaimSubmitted",
            "args": {
                "0":
"1",
                "1":
"1",
                "2":
"0x5B38Da6a701c568545d
CfcB03FcB875f56beddC4"
            ,
                "3":
"7",
                "4":
"Hospital Expenses"
            }
        }
    ]
```

```
[
    {
        "_type": "log",
        "address":
            "0xAc40c9C8dADE7B9CF37
            aeBB49Ab49485eBD3510d",
        ,
        "blockHash":
            "0xc33a452c15857e1d906
            6b85501c7e5a1f56ef95de
            37554782d9c68beed3ad51
            c",
        "blockNumber": 50,
        "data":
            "0x00000000000000000000
            000000000000000000000000
            000000000000000000000000
            10000000000000000000000000
            000000000000000000000000
            00000000000000000000000010
            00000000000000000000000000
            00000000000000000000000000
            00000000000000000000000000
            0000000000000000000000007000
            00000000000000000000000000
            00000000000000000000000000
            00000000000000000000000000
            00000000000000000000000000
            00000000000000000000000000
            00000000000000011486f737
            06974616c20457870656e7
            36573000000000000000000000
            00000000000000",
        "index": 1,
        "topics": [

            "0xd91ad459d6781ac1a94
            eb5c677e1aa55ab8d256ca
            e923e543a7e969235a041c
            a",

            "0x000000000000000000000000"
```

```
000005b38da6a701c56854
5dcfcb03fcb875f56beddc
4"
    ],
    "transactionHash":
"0x906e7095052513e3d5e
90688f08c9c7ace091aa4a
ed1838d30d0da771ca4b9c
5",

"transactionIndex": 0
    }
]
```

>

```
[vm]
from: 0xAb8...35cb2
to: InsuranceContract.approveClaim(uint256) 0xAc4...3510d
value: 0 wei
data: 0x46e...00001
logs: 1
hash: 0x283...70d02
```

status	1 Transaction mined and execution succeed
transaction hash	0x283af6eb4bfde32f3234 cda4a6c013ea509950fcce 49445b470dfccc73770d02
block hash	0x07dbe0a4f3970d89de21 3f41287fe08c41b0b10d78 29b632e135f14a70fd00f5
block number	52
from	0xAb8483F64d9C6d1EcF9b 849Ae677dD3315835cb2
to	InsuranceContract.appr oveClaim(uint256) 0xAc40c9C8dADE7B9CF37a EBb49Ab49485eBD3510d

transaction cost	49890 gas
execution cost	28686 gas
output	0x
decoded input	<pre>{ "uint256 _claimId": "1" }</pre>
decoded output	<pre>{}</pre> <pre>[{ "from": "0xAc40c9C8dADE7B9CF37 aEBb49Ab49485eBD3510d" , "topic": "0x638d892b8630b775fea 5d7a52b209e8bfa1d90c32 b57cd37208ac84d4bcd6dc b", "event": "ClaimApproved", "args": { "0": "1", "1": "1" } }]</pre>
logs	

raw logs

```
[
  {
    "_type": "log",
    "address":
"0xAc40c9C8dADE7B9CF37
aEBb49Ab49485eBD3510d"
  ,
    "blockHash":
"0x07dbe0a4f3970d89de2
13f41287fe08c41b0b10d7
829b632e135f14a70fd00f
5",
    "blockNumber": 52,
    "data":
"0x00000000000000000000
0000000000000000000000
0000000000000000000000
1000000000000000000000
0000000000000000000000
0000000000000000000001"
  ,
    "index": 1,
    "topics": [

"0x638d892b8630b775fea
5d7a52b209e8bfa1d90c32
b57cd37208ac84d4bcd6dc
b"

    ],
    "transactionHash":
"0x283af6eb4bfde32f323
4cda4a6c013ea509950fcc
e49445b470dfccc73770d0
2",

    "transactionIndex": 0
  }
]
```

>

[illegible]

Thank You

-NEELESHWARI MEHRA